



ugr

Universidad
de Granada

SEMINARIO S-sonido

MANEJO DEL SONIDO CON R

Autor:

Mario Casas Pérez

1. Estructura de un archivo de sonido WAV y manejo de sonido R	3
2. Script para probar las funciones implementadas	4
3. Bibliografía	9

1. Estructura de un archivo de sonido WAV y manejo de sonido R

WAV (formato de archivo en forma de onda), es un subconjunto de la especificación de formato de archivo de intercambio de recursos (RIFF), indicando que está estructurado en *chunks* o bloques de datos. Cada uno de estos bloques tiene información concreta que define tanto los metadatos como los datos de sonido. La estructura básica se puede definir como sigue:

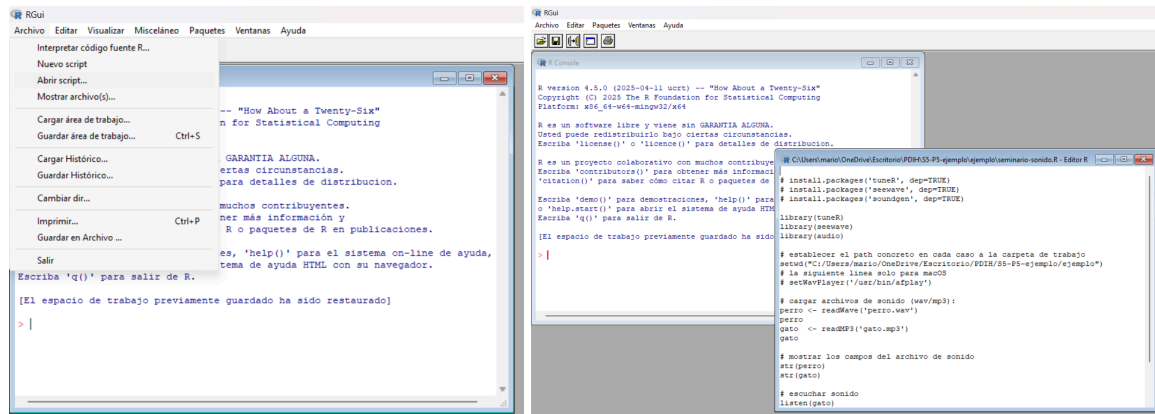
- **RIFF Header:**
 - **Identificador RIFF:** los primeros 4 byte del archivo indican que se trata de un archivo RIFF.
 - **Tamaño del archivo:** luego se especifica el tamaño total del dato restante en el archivo.
 - **Identificador WAVE:** luego se tiene 4 byte adicionales que indican que el contenedor se usa para audio.
- **Chunk FMT("fmt"):** este bloque será el que defina las propiedades del formato de audio. Para un audio en PCM (sin comprimir), el bloque tendrá normalmente 16 bytes. Entre los parámetros tendríamos:
 - **Audio format:** un código que indica el método de codificación.
 - **Número de canales:** indica si el audio es mono (1), estéreo (2) o más.
 - **Frecuencia de muestreo (Sample Rate).**
 - **Byte rate:** la velocidad de datos en bytes por segundo.
- **Block Align:** número de byte por bloque de muestra
- **Bit per Sample:** resolución de audio donde normalmente serán 16, 24 o 32 bits.
- **Chunk data("data"):** son los datos reales del audio, la cabecera tiene:
 - **Identificador "data":** 4 bytes que marcan el inicio de los datos de sonido.
 - **Tamaño del chunk:** cantidad de datos en bytes que siguen.
 - **Muestras de audio:** son los datos en sí, presentados en forma secuencial. Si el archivo es estéreo, las muestras se intercalan.
- **Chunks adicionales:** aunque depende un poco del software que genere el archivo, podría ser que hubiera otros bloques, como por ejemplo *Chunk "LIST"* (información adicional, etiquetas o metadatos) o *Chunk "fact"* (en caso de formatos comprimidos, donde se suele incluir información auxiliar).

La estructura presentada permite el almacenamiento eficiente de audio sin pérdida y meter varios metadatos útiles para la reproducción y control del sonido.

Para el manejo del sonido en R tenemos varios paquetes especializados para el análisis y control del sonido, donde *tuneR* y *seewave* son dos de los más populares, ayudando a la importación, visualización y análisis de archivos WAV, donde destaca la lectura y exploración del archivo, la visualización y análisis de datos de audio, el análisis espectral, la transformada rápida de Fourier (FFT), entre otros.

2. Script para probar las funciones implementadas

Para construir el script y probar las diversas funciones implementadas en el guión, vamos a usar el lenguaje de programación R. Crearemos un fichero *seminario-sonido.R* y probaremos las distintas funciones mostradas.

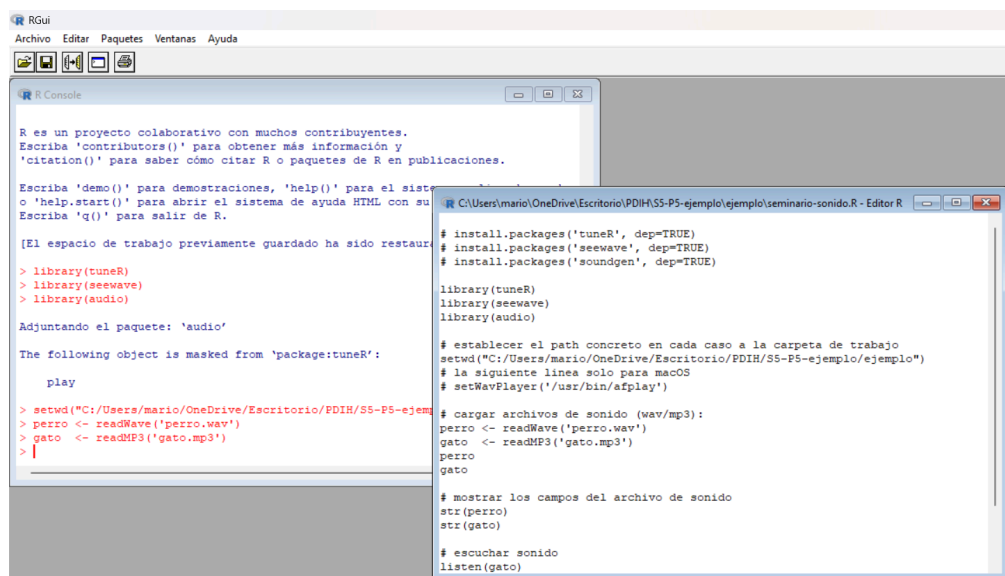


Ahora, pasamos a mostrar las funcionalidades del script para completar este seminario.

1. Leer dos ficheros de sonido (WAV o MP3) de unos pocos segundos de duración cada uno.

Vamos a leer los ficheros donde se escucha un perro ladrando (en formato WAV) y un gato maullando (en formato MP3). Para ello, leeremos los ficheros de sonido con el método *readWave* y *readMP3*, el cual carga el sonido de estos.

```
perro <- readWave('perro.wav')
gato <- readMP3('gato.mp3')
```



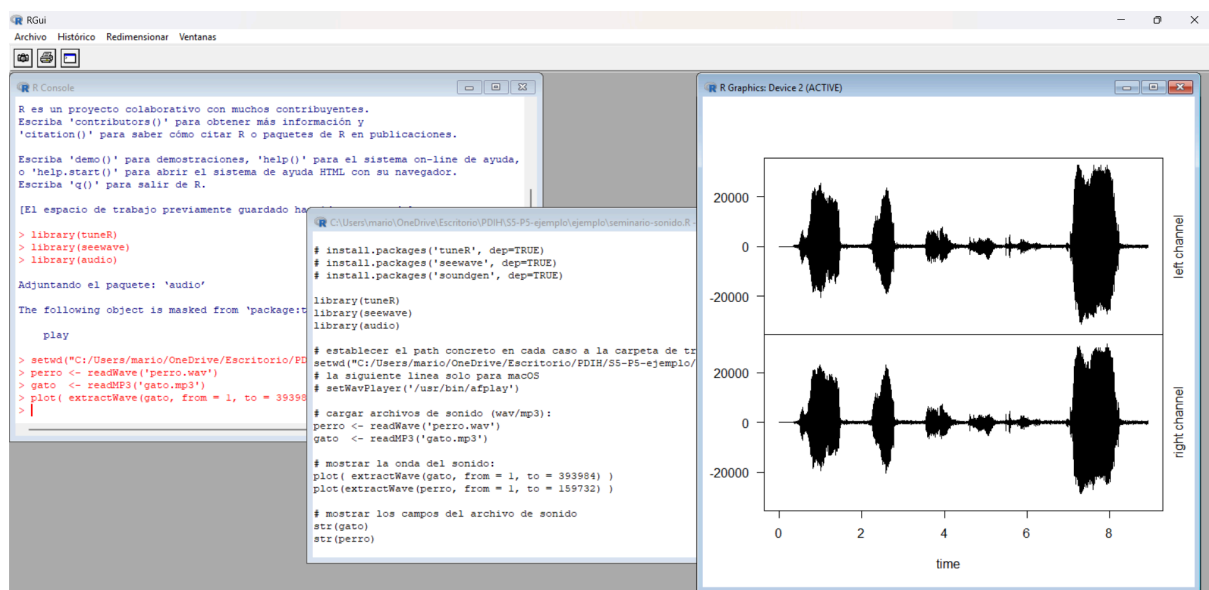
2. Dibuja la forma de onda de ambos sonidos.

Para dibujar la onda de sonido, usaremos la función *plot*.

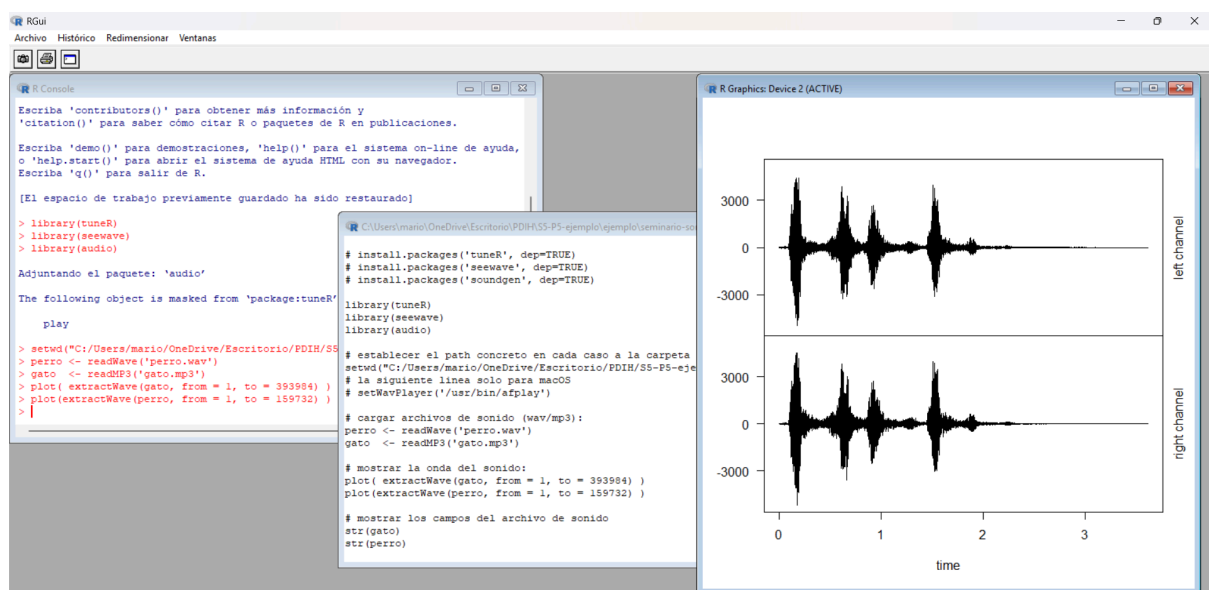
```
plot(extractWave(gato, from = 1, to = 393984))  
plot(extractWave(perro, from = 1, to = 159732))
```

Las ondas de los sonidos del fichero del gato maullando y el perro ladrando son las siguientes.

- Gato



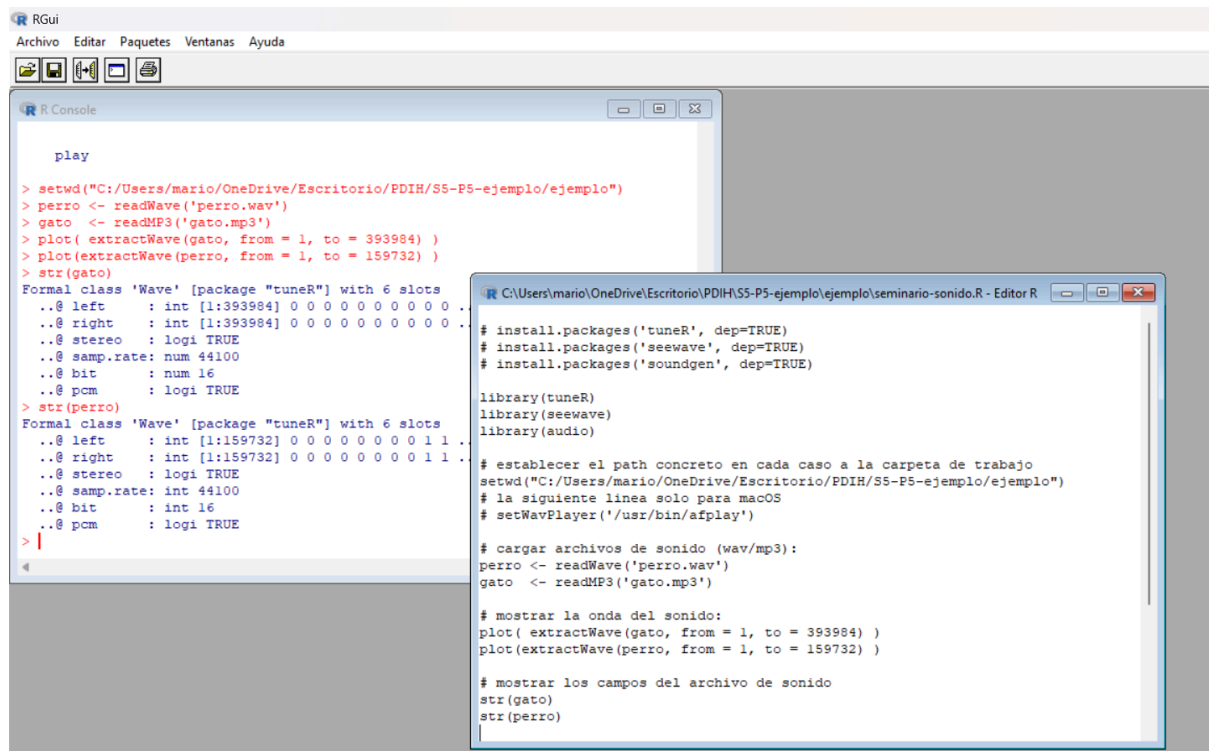
- Perro



3. Obtener la información de las cabeceras de ambos sonidos.

Para inspeccionar las cabeceras de ambos sonidos, usaremos el método *str*.

```
str(gato)
str(perro)
```



4. Unir ambos sonidos en uno nuevo.

Ahora pasamos a la unión del maullido del gato con el ladrido del perro, para ello, usaremos la función *pastew*.

```
GatoPerro <- pastew(perro, gato, output="Wave")
```

Vamos a ver los datos del gato, del perro y de la unión de ambos con dicho método. Para ello, ejecutaremos lo siguiente.

```
gato
perro
GatoPerro
```

```

> GatoPerro <- pastew(perro, gato, output="Wave")
> gato

Wave Object
  Number of Samples:      393984
  Duration (seconds):     8.93
  Samplingrate (Hertz):   44100
  Channels (Mono/Stereo): Stereo
  PCM (integer format):   TRUE
  Bit (8/16/24/32/64):    16

> perro

Wave Object
  Number of Samples:      159732
  Duration (seconds):     3.62
  Samplingrate (Hertz):   44100
  Channels (Mono/Stereo): Stereo
  PCM (integer format):   TRUE
  Bit (8/16/24/32/64):    16

> GatoPerro

Wave Object
  Number of Samples:      553716
  Duration (seconds):     12.56
  Samplingrate (Hertz):   44100
  Channels (Mono/Stereo): Mono
  PCM (integer format):   TRUE
  Bit (8/16/24/32/64):    16

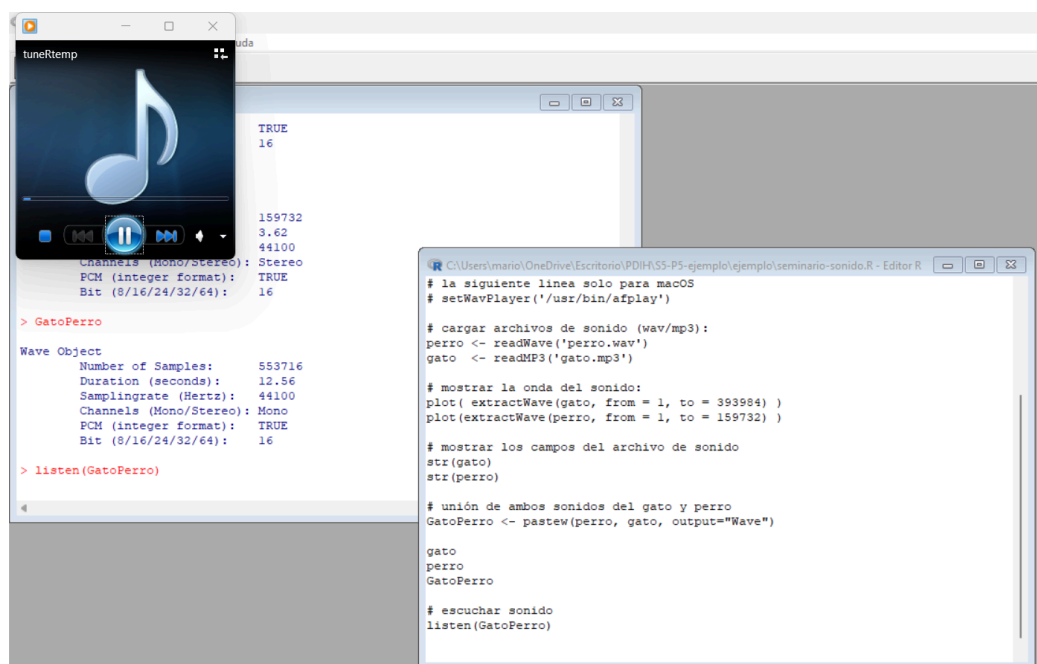
> |

```

5. Reproducir la señal obtenida y almacenarla como un nuevo fichero WAV, denominado “unidos.wav”.

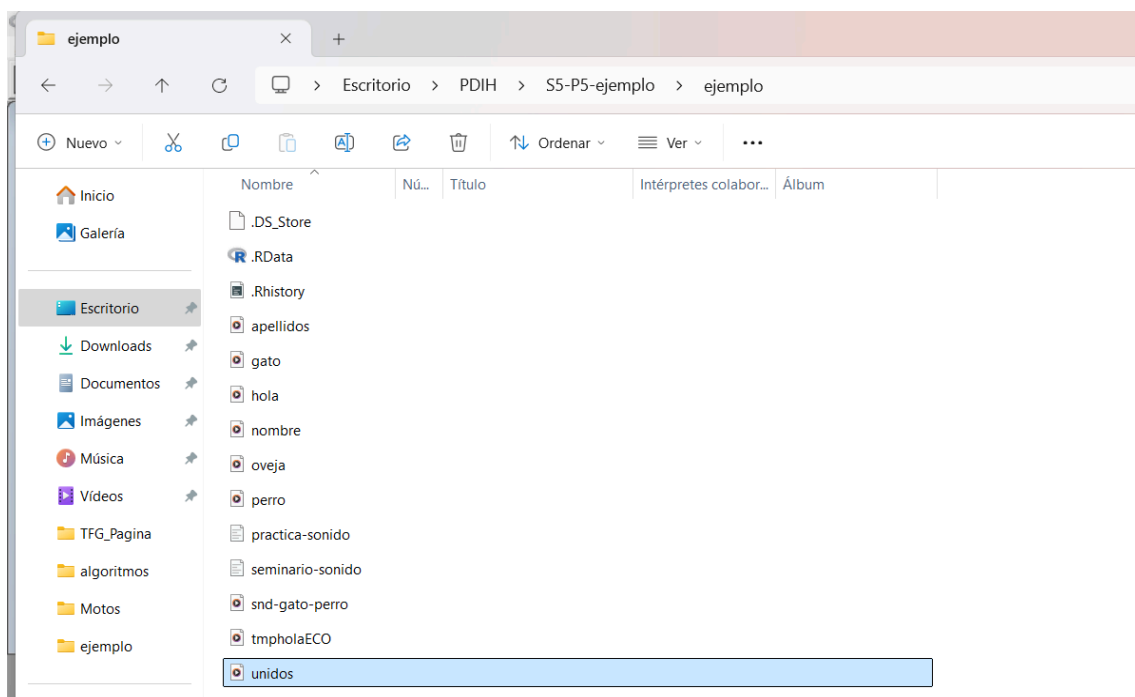
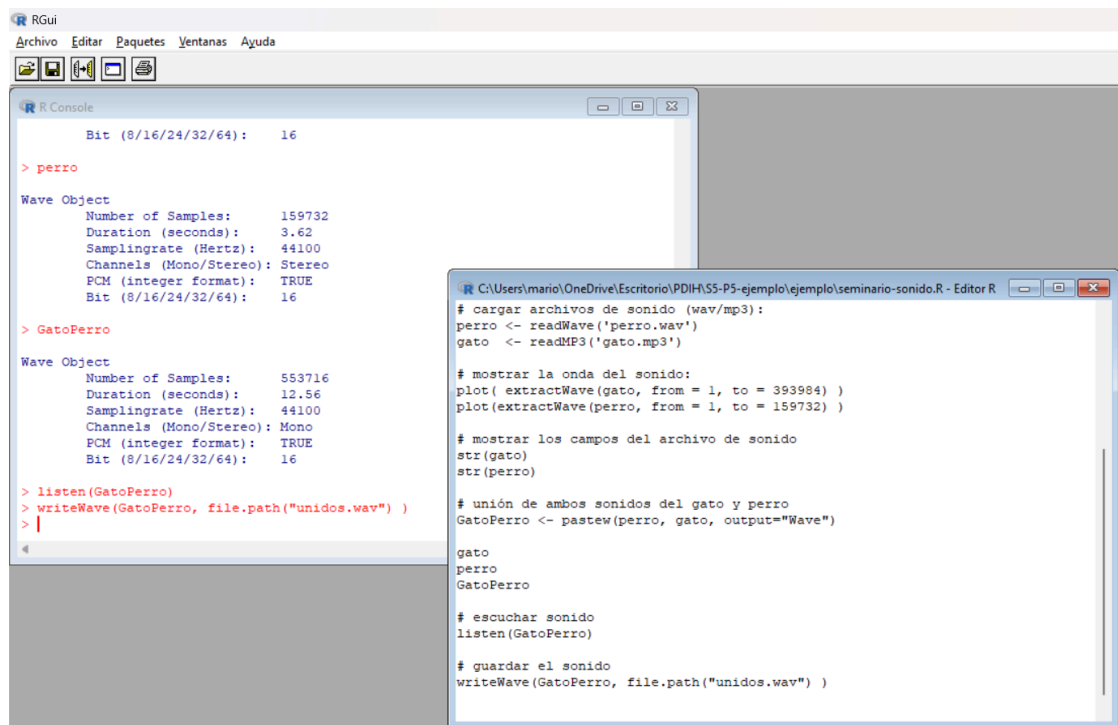
Para escuchar el sonido donde unimos el gato maullando con el perro ladrando, usaremos la función *listen*.

```
listen(GatoPerro)
```



Para guardar el sonido obtenido en el formato que deseemos en disco, usaremos la función *writeWave*. Se guardará en la ruta que se especificó al principio del script (setwd("ruta"))

```
writeWave(GatoPerro, file.path("unidos.wav") )
```



3. Bibliografía

https://swad.ugr.es/swad/tmp/YI/OWc_vD3eFrhbhu90Ev2Jj6I7FgtwvIBaMsXtcMTP0/S-sonido.pdf

<https://docs.fileformat.com/es/audio/wav/>

<https://www.movavi.com/es/learning-portal/wav-file.html>